

ISOM 671

Managing Big Data

RAJIV GARG

Associate Professor

Information Systems & Operations Management

September 11, 2023

Expectations!

- **Stay safe and keep others safe:**
 - If you have covid like symptoms, test negative, and are attending class, please maintain distance for safety of others.
- **Discussion:**
 - Raise hand to ask any questions
 - We may have small group discussions.
- **Quiz:**
 - There may be a 5-minute quiz before class on some days, and it will be administered on Canvas (no paper). You need to make sure your canvas account is accessible during class time.
- **Be courteous:**
 - I don't mind if you are speaking to someone else or doing something else on your computer as long as you are not distracting anyone else in the class. If you are the reason for distraction, I will have to ask you to leave the class.

• **PLEASE USE NAMECARDS**



Updates

- AWS Data Analytics Workshop
 - 9/29 12:00PM to 3PM (**subject to change**)
 - Goals (hands-on):
 - Overview of AWS
 - Learn to connect to RStudio on AWS
 - Bring data from local machine or S3 for analysis
 - Run a .R script to analyze the data
 - After the hands-on you can BYO data and R code to explore

• Organized by Dr. Rajiv Garg, Ron Harris, and AWS@Emory Team (Circe and Troy)



Upcoming Deadlines

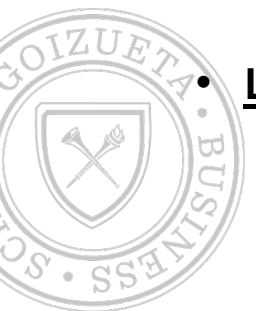
▼ Assignments	
	Assignment 1: ER Model and Normalization Due Sep 12 at 8:30am 30 pts
	Assignment 2: SQL Due Sep 14 at 8:30am 50 pts
	Assignment 3: Loading Data in Relational Databases Due Sep 21 at 8:30am 20 pts



Assignment 2 (5%): Due 9/14 (by 8:30AM)

- Write SQL queries to answer ALL questions.
 - There are 25 numbered questions (2 point for each correct answer)
 - Questions based on an online database, guitar-shop database, and hypothetical databases.
 - Please answer them all and submit your assignment by uploading it to the course canvas page. TWO files need to be submitted:
 - Submit a ".sql" file with all SQL queries and comments identifying the question being answered.
 - Submit a ".pdf" file with all SQL statements, results of the SQL statement (typically copy first 5 rows), and answers to any questions.

• **LATE PENALTY:** 25% for each 24 hours





SQL I

Catch up

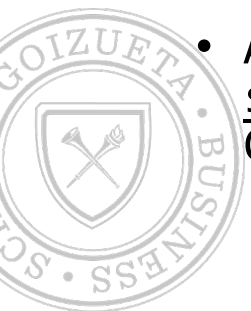


SQL II

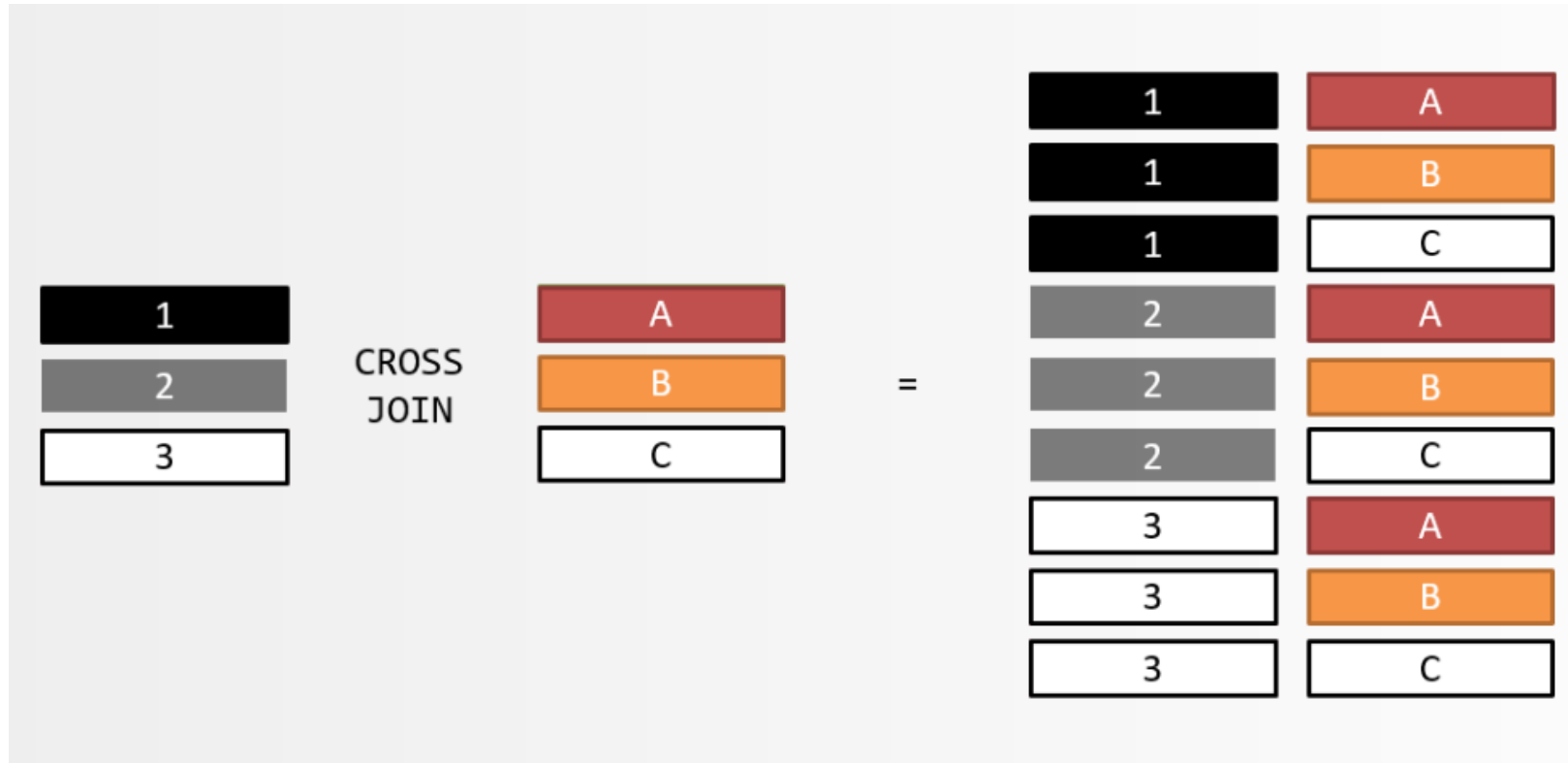
Data Querying

JOIN data from multiple tables

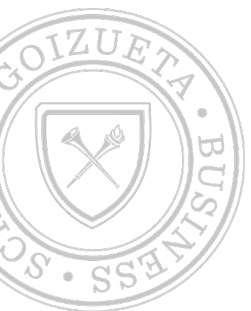
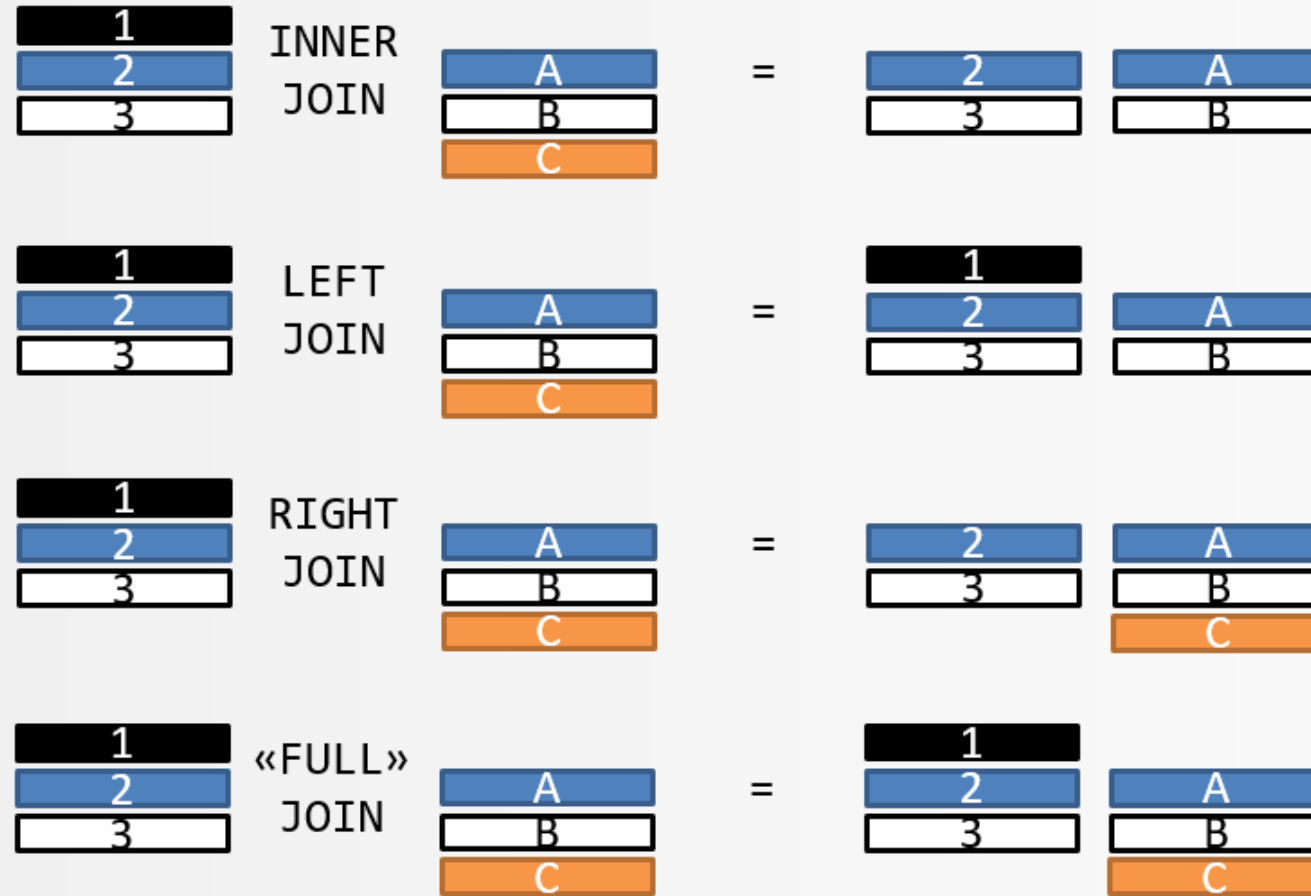
- A join lets you combine columns from two or more tables into a single result set.
 - A join between tables returns all unique combinations of their tuples which meet some specified join condition.
- For an **inner join**, only those rows that satisfy the join condition(s) are included in the result set.
- An **outer join** retrieves all rows that satisfy the join condition(s), plus unmatched rows in the left or right table.
- A **cross join** joins each row from the first table with each row from the second table. The result set returned by a cross join is known as a Cartesian product.



JOIN: Cross



JOIN: Inner and Outer



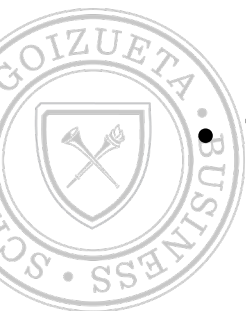
JOIN: Query

- Syntax:

- ```
SELECT select_list
 FROM [table_1_database_name.] table_1 [table_1_alias]
 [INNER] JOIN [table_2_database_name.]table_2 [table_2_alias]
 ON join_condition_1 [join_condition_2 ...]
 [[INNER] JOIN [table_3_database_name.]table_3 [table_3_alias]
 ON join_condition_3 [join_condition_4 ...]]...
```

- A join condition names a column in each of the two tables involved in the particular join and indicates how the two columns should be compared.
  - In most cases, you use the equal operator to retrieve rows with matching columns. However, you can also use any of the other comparison operators in a join condition.

- Tables are typically joined on the relationship between the primary key in one table and a foreign key in the other table.



# JOIN (INNER)

- **JOIN:**

```
SELECT c.first_name, c.last_name, a.address
FROM customer AS c
JOIN address AS a;
```

Cartesian product:  
every customer with  
every address

- **INNER JOIN:**

```
SELECT c.first_name, c.last_name, a.address
FROM customer AS c
INNER JOIN address AS a
ON c.address_id = a.address_id;
```

Limits join to  
customers with  
address\_id

```
SELECT c.first_name, c.last_name, a.address, ct.city
FROM customer AS c
INNER JOIN address AS a
ON c.address_id = a.address_id
INNER JOIN city AS ct
ON a.city_id = ct.city_id;
```

Joining across three  
tables



# JOIN (OUTER)

- Inner join gets rows that are matched on a criteria in both tables
  - Thus, only rows where (c.address\_id = a.address\_id) are shown.
- What if you want to see the rows from customer (c) table even if that customer has address\_id as NULL?

```
SELECT c.first_name, c.last_name, a.address
FROM customer AS c
LEFT OUTER JOIN address AS a
ON c.address_id = a.address_id;
```

Left outer join: get rows from customer table even if they don't have matching address\_id

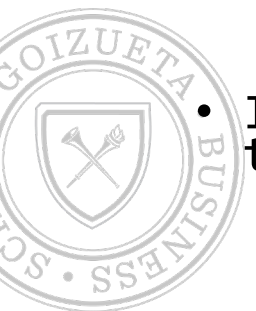
```
SELECT c.first_name, c.last_name, a.address
FROM customer AS c
RIGHT OUTER JOIN address AS a
ON c.address_id = a.address_id;
```

Right outer join: get rows from address table even if they don't have matching address\_id



# INNER JOIN: creating query

- A join condition can include two or more conditions connected by AND or OR.
- When you run a SELECT statement against one database, you can join to a table in another database.
  - To do that, you must prefix the table name in the other database with the name of that database. You also need to have appropriate permissions.
- You can think of a multi-table join as a series of two-table joins proceeding from left to right.
- To code a self-join, you must use aliases for the tables, and you must qualify each column name with the alias.
- Instead of coding a join condition in the FROM clause, you can code it in the WHERE clause along with any search conditions as an implicit join.



# INNER JOIN: Additional Examples

```
SELECT invoice_number, vendor_name
FROM vendors v, invoices i
WHERE v.vendor_id = i.vendor_id
ORDER BY invoice_number
```

Identify vendor  
and invoice from  
two tables

```
SELECT invoice_number, vendor_name
FROM vendors INNER JOIN invoices
 ON vendors.vendor_id = invoices.vendor_id
ORDER BY invoice_number
```

```
SELECT customer_first_name, customer_last_name
FROM customers c JOIN employees e
 ON c.customer_first_name = e.first_name
 AND c.customer_last_name = e.last_name
```

Identify customers  
that are also  
employees

```
SELECT vendor_name, customer_last_name, customer_first_name,
 vendor_state AS state, vendor_city AS city
FROM vendors v
 JOIN customers c
 ON v.vendor_zip_code = c.customer_zip
ORDER BY state, city
```

Identify vendors  
and customers in  
the same zip code

```
SELECT DISTINCT v1.vendor_name, v1.vendor_city,
 v1.vendor_state
FROM vendors v1 JOIN vendors v2
 ON v1.vendor_city = v2.vendor_city AND
 v1.vendor_state = v2.vendor_state AND
 v1.vendor_name <> v2.vendor_name
ORDER BY v1.vendor_state, v1.vendor_city
```

Identify distinct  
vendors in the  
same city, state



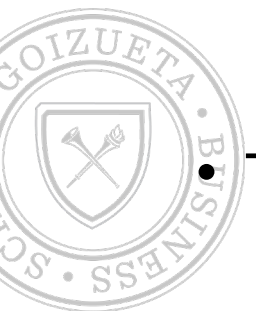
# OUTER JOIN: creating query

- When you use a left outer join, the result set includes all the rows from the first (or left) table.
  - Similarly, when you use a right outer join, the result set includes all the rows from the second (or right) table.

- Syntax:

- ```
SELECT select_list
FROM table_1
    {LEFT|RIGHT} [OUTER] JOIN table_2
        ON join_condition_1
    [{LEFT|RIGHT} [OUTER] JOIN table_3
        ON join_condition_2]...
```

- The OUTER keyword is optional.



OUTER JOIN: Additional Examples

```
SELECT department_name, e.department_number, last_name
FROM departments d
     RIGHT JOIN employees e
     ON d.department_number = e.department_number
ORDER BY department_name
```

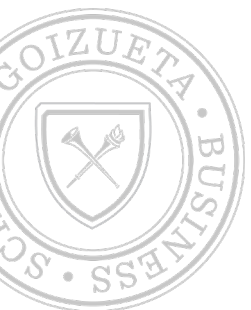
Get all employee names and department name (if available)

```
SELECT department_name, last_name, project_number
FROM departments d
     LEFT JOIN employees e
     ON d.department_number = e.department_number
     LEFT JOIN projects p
     ON e.employee_id = p.employee_id
ORDER BY department_name, last_name
```

Get all employee names and department name (if available) and employee project number (if available)

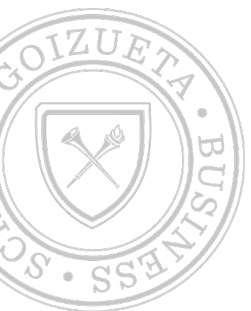
```
SELECT department_name, last_name, project_number
FROM departments d
     JOIN employees e
     ON d.department_number = e.department_number
     LEFT JOIN projects p
     ON e.employee_id = p.employee_id
ORDER BY department_name, last_name
```

Get all employee names WITH department name and employee project number (if available)



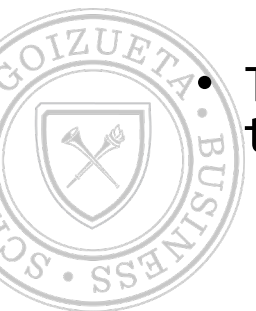
CROSS JOIN: creating query

- A cross join joins each row from the first table with each row from the second table.
- Syntax:
 - `SELECT select_list
FROM table_1 CROSS JOIN table_2`
 - or
 - `SELECT select_list
FROM table_1, table_2`



UNION

- A union combines the result sets of two or more `SELECT` statements into one result set.
 - The column names in the final result set are taken from the first `SELECT` clause.
- Each result set must return the same number of columns, and the corresponding columns in each result set must have compatible data types.
- By default, a union eliminates duplicate rows. If you want to include duplicate rows, code the `ALL` keyword.
- To sort the rows in the final result set, code an `ORDER BY` clause after the last `SELECT` statement.
 - This clause must refer to the column names assigned in the first `SELECT` clause.



UNION: creating query

- Syntax of the union operation:
 - `SELECT_statement_1`
 - `UNION [ALL]`
 - `SELECT_statement_2`
 - `[UNION [ALL]`
 - `SELECT_statement_3] . . .`
 - `[ORDER BY order_by_list]`



UNION

- What if you want to combine data from multiple tables into one table?
 - JOIN allows you to combine two datasets side-by-side, but UNION allows you to stack one dataset on top of the other.

- Create a union of two different staff tables:

- `SELECT * FROM staff`
- **UNION**
- `SELECT * FROM staff;`

UNION combines distinct rows in sorted order

- Place the union into a new table:

- `CREATE TABLE staff_new AS`
- `SELECT * FROM staff`
- **UNION ALL**
- `SELECT * FROM staff;`

Output of a UNION can also be inserted into a new table

UNION ALL combines ALL rows



UNION

- List movies that have duration 120, 130, or 140 minutes (*you can use OR in SELECT*)

```
SELECT * FROM film WHERE length = 120
```

UNION

```
SELECT * FROM film WHERE length = 130
```

UNION

```
SELECT * FROM film WHERE length = 140;
```

UNION can be compounded
and processed in order

*Think: (UNION, UNION ALL)
vs (UNION ALL, UNION)*

- Combine the dataset and order by columns (show all actor and customer names where first name starts with J and last name starts with D).

```
SELECT a.first_name fname, a.last_name lname FROM actor a  
WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%'
```

UNION

```
SELECT c.first_name, c.last_name FROM customer c  
WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%'  
ORDER BY fname, lname;
```



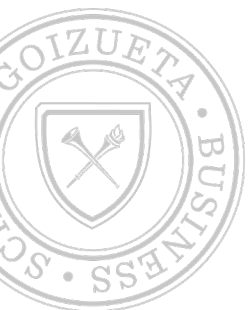
UNION: Additional Examples

```
SELECT 'Active' AS source, invoice_number, invoice_date, invoice_total
FROM active_invoices
WHERE invoice_date >= '2011-06-01'
UNION
SELECT 'Paid' AS source, invoice_number, invoice_date, invoice_total
FROM paid_invoices
WHERE invoice_date >= '2011-06-01'
ORDER BY invoice_total DESC
```

Combine invoices from
two different tables
(active and paid)

```
SELECT department_name AS dept_name, d.department_number AS d_dept_no,
       e.department_number AS e_dept_no, last_name
FROM departments d
      LEFT JOIN employees e
            ON d.department_number = e.department_number
UNION
SELECT department_name AS dept_name, d.department_number AS d_dept_no,
       e.department_number AS e_dept_no, last_name
FROM departments d
      RIGHT JOIN employees e
            ON d.department_number = e.department_number
ORDER BY dept_name
```

Combine the results of
left join and right join of
employee and
department tables



Summary Query (Aggregate functions)

- A summary query is a `SELECT` statement that includes one or more aggregate functions.
 - Summary queries provide important analytical functions to DBMSs.
- The expression you specify for the `AVG` and `SUM` functions must result in a numeric value. The expression for the `MIN`, `MAX`, and `COUNT` functions can result in a numeric, date, or string value.
- By default, all values are included in the calculation regardless of whether they're duplicated. If you want to omit duplicate values, code the `DISTINCT` keyword. All of the aggregate functions except for `COUNT(*)` ignore null values.

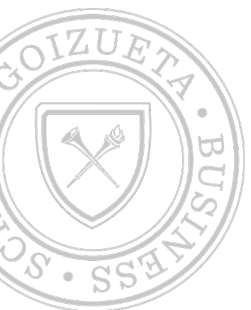


Aggregate Functions

Function syntax	Result
AVG([ALL DISTINCT] expression)	The average of the non-null values in the expression
SUM([ALL DISTINCT] expression)	The total of the non-null values in the expression
MIN([ALL DISTINCT] expression)	The lowest non-null value in the expression
MAX([ALL DISTINCT] expression)	The highest non-null value in the expression
COUNT([ALL DISTINCT] expression)	The number of non-null values in the expression
COUNT(*)	The number of rows selected by the query

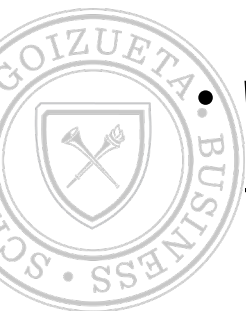
- Example:

```
SELECT COUNT(DISTINCT vendor_id) AS number_of_vendors,  
       COUNT(vendor_id) AS number_of_invoices,  
       ROUND(AVG(invoice_total), 2) AS avg_invoice_amt,  
       SUM(invoice_total) AS total_invoice_amt  
FROM invoices  
WHERE invoice_date > '2011-01-01'
```



GROUP BY

- The GROUP BY clause groups the rows of a result set based on one or more columns or expressions.
- If you include aggregate functions in the SELECT clause, the aggregate is calculated for each group specified by the GROUP BY clause.
- The HAVING clause specifies a search condition for a group or an aggregate.
 - MySQL applies this condition after it groups the rows that satisfy the search condition in the WHERE clause.
- When a GROUP BY clause is used, the SELECT clause can include the columns used for grouping, aggregate functions, and expressions that result in a constant value.



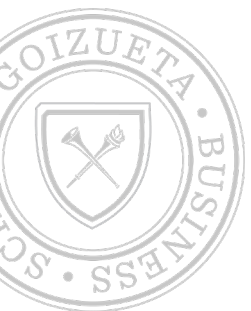
GROUP BY and HAVING

- The syntax of the GROUP BY and HAVING clauses:

- `SELECT select_list`
`FROM table_source`
`[WHERE search_condition]`
`[GROUP BY group_by_list]`
`[HAVING search_condition]`
`[ORDER BY order_by_list]`

- Example:

```
SELECT vendor_id, ROUND(AVG(invoice_total), 2) AS average_invoice_amount
FROM invoices
GROUP BY vendor_id
HAVING AVG(invoice_total) > 2000
ORDER BY average_invoice_amount DESC
```



GROUP BY: HAVING

- Groups similar data together and enables aggregation functions (max, min, avg, sum):

- Query the number of rentals by each customer_id:

```
SELECT customer_id, count(*)  
FROM rental  
GROUP BY customer_id;
```

Counts in each group

- Get high frequency renters:

```
SELECT customer_id, count(*)  
FROM rental  
GROUP BY customer_id  
HAVING count(*)>40;
```

Note: WHERE will not work here because it is executed before GROUP BY



WHERE vs HAVING

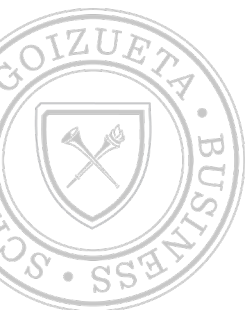
- When you include a WHERE clause in a SELECT statement that uses grouping and aggregates, the search condition is applied before grouping the rows and calculating the aggregates.
- When you include a HAVING clause in a SELECT statement that uses grouping and aggregates, the search condition is applied after grouping the rows and calculating the aggregates.
- A WHERE clause can refer to any column in the base tables.
- A HAVING clause can only refer to a column included in the SELECT clause.
- A WHERE clause can't contain aggregate functions.
- A HAVING clause can contain aggregate functions.
- You can use the AND and OR operators to code compound search conditions in both HAVING and WHERE clauses.



GROUP BY: aggregation functions

- Using aggregation functions (max, min, avg, sum):

```
SELECT customer_id,  
       MAX(amount) max_amt,  
       MIN(amount) min_amt,  
       AVG(amount) avg_amt,  
       SUM(amount) tot_amt,  
       count(*) num_payments  
FROM payment  
GROUP BY customer_id;
```

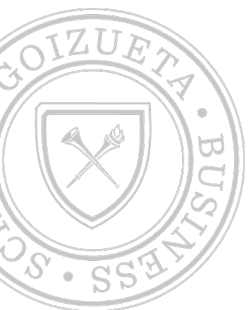


GROUP BY: multiple columns

- Grouping on multiple columns – actor and film rating (number of movies an actor has acted in a specific rated film):

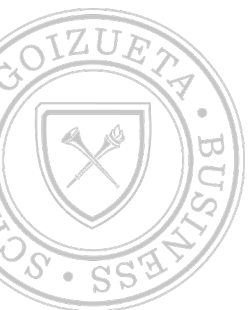
```
SELECT fa.actor_id, f.rating, count(*) num_films
FROM film_actor AS fa
      INNER JOIN film AS f
      ON fa.film_id = f.film_id
GROUP BY fa.actor_id, f.rating
ORDER BY 1, 2;
```

Grouping on multiple
columns



Speed Test! (ungraded)

- Write queries for the following (accuracy + speed matters):
 1. Display the most frequently rented movies in descending order.
 2. For a marketing campaign display all customer names and emails in Canada
 3. Display the total payment processed by each staff member during each month in 2005.
 4. List the top five genres in gross revenue in descending order.



Speed Test! (solutions)

```
#Display the most frequently rented movies in descending order.
SELECT title, COUNT(title) as 'Rentals'
FROM film
JOIN inventory
ON (film.film_id = inventory.film_id)
JOIN rental
ON (inventory.inventory_id = rental.inventory_id)
GROUP BY title
ORDER BY rentals desc;
```

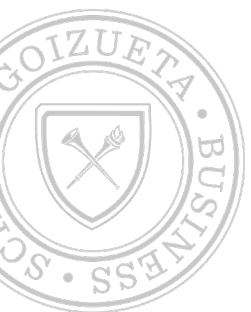
```
#Display the total payment processed by each staff member during each
month in 2005.
SELECT s.first_name, s.last_name, MONTH(p.payment_date),
SUM(p.amount)
FROM staff s
INNER JOIN payment p
ON (s.staff_id = p.staff_id)
WHERE YEAR(p.payment_date) = 2005
GROUP BY s.staff_id, MONTH(p.payment_date)
ORDER BY MONTH(p.payment_date), s.staff_id;
```

```
#For a marketing campaign display all customer names and emails in
Canada
SELECT c.first_name, c.last_name, c.email
FROM customer c
JOIN address a ON (c.address_id = a.address_id)
JOIN city ci ON (a.city_id = ci.city_id)
JOIN country ctr ON (ci.country_id = ctr.country_id)
WHERE ctr.country = 'canada';
```

```
#List the top five genres in gross revenue in descending order.
SELECT c.name AS 'Genre', SUM(amount) AS 'Total Sales'
FROM payment p
JOIN rental r
ON (p.rental_id = r.rental_id)
JOIN inventory i
ON (r.inventory_id = i.inventory_id)
JOIN film_category fc
ON (i.film_id = fc.film_id)
JOIN category c
ON (fc.category_id = c.category_id)
GROUP BY c.name
ORDER BY SUM(amount) DESC;
```



SQL & Text



Text Analytics

- While SQL is not the optimal choice for NLP and sophisticated text analytics, it does offer multiple benefits:
 - Data in RDBMS (structured table form)
 - SQL is simple for quantification of insights in text data
 - E.g. number of movies with description containing “scientist”
 - String functions can be nested for deeper analysis
 - Offers a rule-based analysis that is easy to understand
 - Fast when you know what you are looking for



Text Length()

- `select title, description, length(description)`
 - `from film`
 - `where description regexp "scientist";`
-
- `select length(description), count(*)`
 - `from film`
 - `group by 1`
 - `order by 1 desc;`



Text Parsing (left, right)

- `Left()`: returns characters from the left (or beginning) of the string
- `Right()` : returns characters from the right (or end) of the string
- `select description, length(description), left(description, 10), right(description, 10)`
- `from film`
- `where description regexp "scientist";`
- `select left(description, 5), count(*)`
- `from film`
- `group by 1`
- `order by 1 desc;`



Text Parsing (substring)

- `SUBSTRING(string, position, length)`: returns the “length” of characters from “position” in the “string”
- `SUBSTRING_INDEX(string, delimiter, index)`: returns the entire text to the left of the “delimiter” at “index”
- `select description, SUBSTRING(description, 5, 5)`
- `from film;`
- `select description, SUBSTRING_INDEX(description, " ", 2)`
- `from film;`
- `select description, SUBSTRING_INDEX(description, "And", 1)`
- `from film;`



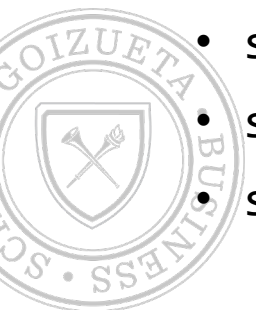
Text Parsing (substring nesting)

- Find text between two words in the description data
- select description,
- **SUBSTRING_INDEX(SUBSTRING_INDEX(description, "And", 1), " a ", -1)**
- from film;
- Select **SUBSTRING_INDEX(SUBSTRING_INDEX(description, "And", 1), " a ", -1),**
count(*)
- from film
- group by 1
- order by 1 desc;



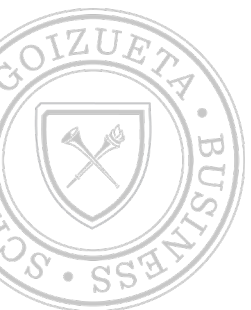
Text Transformation

- `Lower(string)`: converts all characters in a string to lower case
 - `Upper(string)`: converts all characters in a string to upper case
 - `Trim(string)`: removes blank space from beginning and ending of string
 - `Trim(leading/trailing CHAR from string)`: removes CHAR from beginning (leading) or ending (training) of string
-
- `select title, lower(title), upper(title)`
 - `from film;`
-
- `select trim(" I am a short sentence ");`
 - `select trim(leading "*" from "*I am a short sentence*");`
 - `select trim(trailing "*" from "*I am a short sentence*");`



Text Transformation

- `Replace(string, substring, replacement)`: Replace substring with replacement
- `select description, replace(description, "And", "OR")`
- `from film;`



Text Labels (case)

- Assign labels to text and count text of those label types
- select
- case
 - when lower(description) regexp "tale" then "tale"
 - when lower(description) regexp "saga" then "saga"
 - when lower(description) regexp "documentary" then "documentary"
 - when lower(description) regexp "drama" then "drama"
 - else "other"
- end as film_type, count(*)
- from film
- group by 1;



Text Labels (case)

- Assign labels to text and count text of those label types and organize in columns
- select
- count(case when lower(description) regexp "tale" then 1 end) as tale,
- count(case when lower(description) regexp "saga" then 1 end) as tale,
- count(case when lower(description) regexp "documentary" then 1 end) as tale,
- count(case when lower(description) regexp "drama" then 1 end) as tale
- from film;



Text Search - LIKE and REGEXP

- LIKE wildcards

Symbol	Description
%	Matches any string of zero or more characters
_	Matches any single character

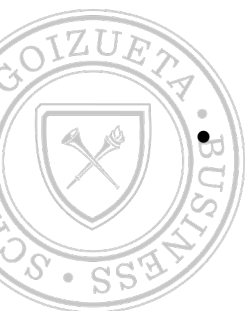
- REGEXP special characters and constructs

Construct	Description
^	Matches the pattern to the beginning of the value being tested
\$	Matches the pattern to the end of the value being tested
.	Matches any single character
[TEXTlist]	Matches any single character listed within the brackets
[TEXT1-TEXT2]	Matches any single character within the given range
	Separates two string patterns and matches either one



Text Search - REGEXP

- `[:class:]`
 - Searches for a class (`[:digit:]`, `[:alpha:]`) of characters in string
 - `SELECT title,`
 - `title REGEXP '[:alpha:]', title REGEXP '[:digit:]'`
 - `FROM film;`
- `{count}`
 - Finds the strings with number of instances of characters preceding the count
 - Finds the strings with `{min,max}` number of instances of characters preceding the count
 - `select description, length(description), description regexp '^.{95}$' from film;`
 - `select description, length(description), description regexp '^.{95,98}$' from film;`



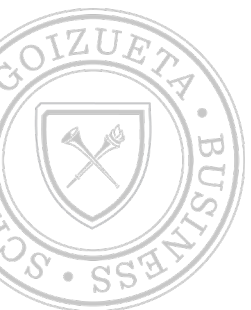
Text Search - REGEXP

- REGEXP_LIKE(): returns 1 if it finds pattern in a string
- REGEXP_INSTR(): returns index of string where pattern is found
- select description,
- description regexp 'scientist',
- REGEXP_LIKE(description, 'scientist'),
- REGEXP_INSTR(description, 'scientist')
- from film;
- select description,
- REGEXP_LIKE(description, 'a '),
- REGEXP_INSTR(description, 'a '),
- REGEXP_INSTR(description, 'a ', 2)
- from film limit 20;



Text Search - REGEXP

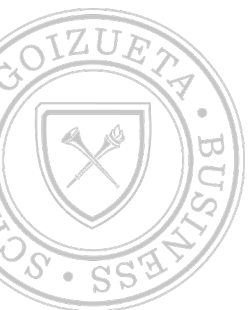
- REGEXP_REPLACE(): replaces the found pattern with another string
- select description,
- REGEXP_LIKE(description, 'a '),
- REGEXP_INSTR(description, 'a '),
- REGEXP_INSTR(description, 'a ', 2),
- REGEXP_REPLACE(description, 'a ', "NEW"),
- REGEXP_REPLACE(description, 'a ', "NEW", 2)
- from film limit 20;



Text Search - REGEXP

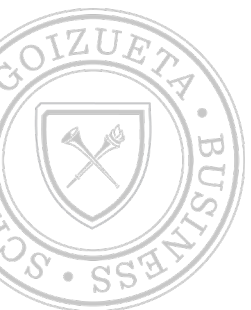
- REGEXP_SUBSTR(): return the pattern found in the string
- select description,
- REGEXP_LIKE(description, '[a-z]+'),
- REGEXP_SUBSTR(description, '[a-z]+'),
- REGEXP_SUBSTR(description, '[a-z]+', 3)
- from film limit 20;

Position to start looking for
the pattern in the string.
Here, starting from 3rd character

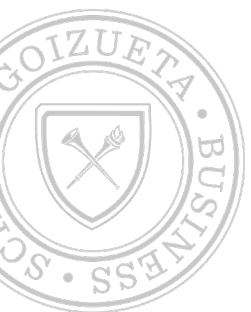


Text Search - REGEXP

- `GROUP_CONCAT()`: combines all values in rows into a list in one row
- `SELECT GROUP_CONCAT(distinct rating) AS ratings`
- `FROM film;`
- `SELECT rating, GROUP_CONCAT(distinct rental_rate ORDER BY rental_rate) AS ratings`
- `FROM film`
- `GROUP BY rating;`



Subqueries



SQL subqueries

- A subquery is a SELECT statement that's coded within another SQL statement.
- A subquery can return a single value, a list of values (a result set that has a single column), or a table of values (a result set that has multiple columns).
- A subquery can be coded, or introduced, anywhere a single value, a list of values, or a table is allowed.
- The syntax for a subquery is the same as for a standard SELECT statement.
 - However, a subquery can't include an ORDER BY clause.
- Subqueries can be nested within other subqueries.



SQL subqueries

- QUERY data from a QUERIED DATA
- Find the name of the customer that was last added into the system (largest customer id)

```
SELECT MAX(customer_id) FROM customer;
```

```
> 599
```

```
SELECT customer_id, first_name, last_name FROM customer WHERE customer_id = 599;
```

- Alternatively:

```
SELECT customer_id, first_name, last_name FROM customer
```

```
WHERE customer_id = (SELECT MAX(customer_id) FROM customer);
```

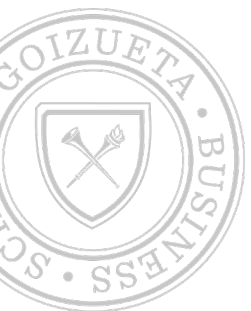


SQL subqueries: correlated

- A correlated subquery is a subquery that is executed once for each row in the main query. In contrast, an uncorrelated subquery is executed only once.
- A correlated subquery refers to a value that's provided by a column in the main query. For each different value that's returned by the main query for that column, the subquery returns a different result.

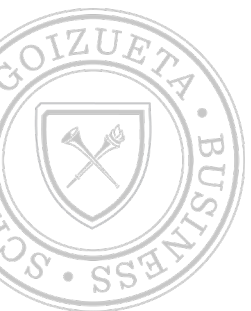
- Example:

```
SELECT vendor_id, invoice_number, invoice_total
FROM invoices i
WHERE invoice_total >
      (SELECT AVG(invoice_total)
       FROM invoices
       WHERE vendor_id = i.vendor_id)
ORDER BY vendor_id, invoice_total
```



SQL subqueries: IN operator

- You can introduce a subquery with the IN operator to provide the list of values that are tested against the test expression.
- When you use the IN operator, the subquery must return a single column of values.
- The syntax of a WHERE clause that uses an IN phrase:
 - `WHERE test_expression [NOT] IN (subquery)`

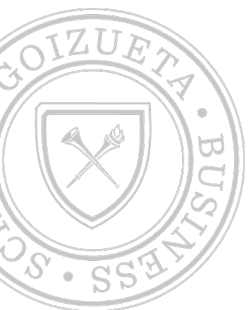


SQL subqueries: IN operator

- Subqueries that don't have any correlation with the main query are called Noncorrelated (scalar) queries:

```
SELECT city_id, city
FROM city
WHERE country_id <> (SELECT country_id FROM country WHERE country = "India");
```

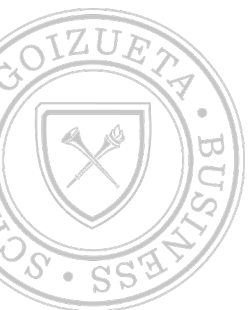
```
SELECT city_id, city
FROM city
WHERE country_id IN (SELECT country_id FROM country WHERE country IN ("India", "Canada", "Mexico"));
```



SQL subqueries: BETWEEN

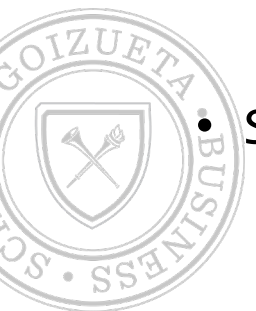
- Subqueries that have correlation with the main query are called Correlated (dependent) queries:

```
SELECT c.first_name, c.last_name  
FROM customer AS c  
WHERE (SELECT sum(p.amount) FROM payment AS p WHERE p.customer_id = c.customer_id)  
BETWEEN 180 AND 240;
```



SQL subqueries: comparison

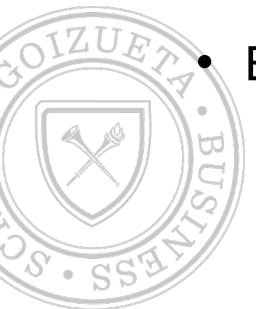
- You can use a comparison operator in a WHERE clause to compare an expression with the results of a subquery.
- If you code a search condition without the ANY, SOME, and ALL keywords, the subquery must return a single value.
- If you include the ANY, SOME, or ALL keyword, the subquery can return a list of values.
 - You can use the ALL keyword to test that a comparison condition is true for all of the values returned by a subquery.
 - You can use the ANY keyword to test that a condition is true for one or more of the values returned by a subquery.
 - The SOME keyword works the same as the ANY keyword.
- Syntax:
 - WHERE expression comparison_operator [SOME|ANY|ALL] (subquery)



SQL subqueries: EXISTS

- You can use the EXISTS operator to test that one or more rows are returned by the subquery.
 - You can use the NOT EXISTS operator to test that no rows are returned by the subquery.
- When you use these operators with a subquery, it doesn't matter what columns you specify in the SELECT clause. As a result, you typically just code an asterisk (*).
- Syntax: WHERE [NOT] EXISTS (subquery)
- Example:

```
SELECT vendor_id, vendor_name, vendor_state
FROM vendors
WHERE NOT EXISTS
  (SELECT *
   FROM invoices
   WHERE vendor_id = vendors.vendor_id)
```

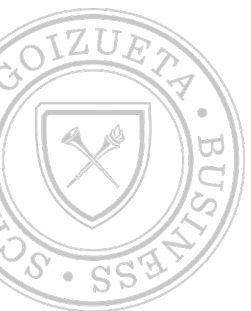


SQL subqueries: FROM

- A subquery that's coded in the FROM clause returns a result set that can be referred to as an inline view.
- When you code a subquery in the FROM clause, you must assign an alias to it.
 - You can use that alias just as you would any other table name or alias.

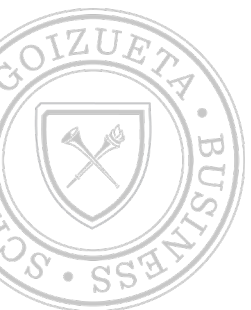
- Example:

```
SELECT vendor_state, MAX(sum_of_invoices) AS max_sum_of_invoices
FROM
(
    SELECT vendor_state, vendor_name,
           SUM(invoice_total) AS sum_of_invoices
    FROM vendors v JOIN invoices i
      ON v.vendor_id = i.vendor_id
    GROUP BY vendor_state, vendor_name
) t
GROUP BY vendor_state
```



SQL subqueries

- SQL subqueries allow you to create an extremely large SQL code for almost any complex data querying and processing task.
- To tackle any complex task, think of divide and conquer:
 - **Divide:** data processing into sequential steps
 - **Code:** query for each step
 - **Combine:** queries in a nested structure to combine the output



SQL WITH Clause

SQL Subqueries



WITH Subqueries

- Use a WITH clause that has one or more comma-separated subclauses. Each subclause provides a subquery that produces a result set, and associates a name with the subquery.
- WITH
- cte1 AS (SELECT a, b FROM table1),
- cte2 AS (SELECT c, d FROM table2)
- SELECT b, d FROM cte1 INNER JOIN cte2
- ON cte1.a = cte2.c;

- You can also do recursive coding with “WITH” in SQL. For more information, please see: <https://dev.mysql.com/doc/refman/8.0/en/with.html>

Source: <https://dev.mysql.com/doc/refman/8.0/en/with.html>



SQL Windows



SQL - Windows

- What is a moving average?
- Where is it used?
- Why is it useful?
- How would you estimate it programmatically?

	fiscal_year	sales_employee	sale	total_sales
▶	2016	Alice	150.00	450.00
	2016	Bob	100.00	450.00
	2016	John	200.00	450.00
	2017	Alice	100.00	400.00
	2017	Bob	150.00	400.00
	2017	John	150.00	400.00
	2018	Alice	200.00	650.00
	2018	Bob	200.00	650.00
	2018	John	250.00	650.00



Image source: <https://www.mysqltutorial.org/mysql-window-functions/>

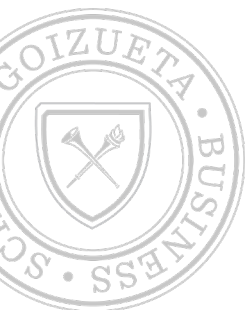
SQL - Windows

- Moving window of data:

```
SELECT Ticker, SUM(DivAmt), AVG(DivAmt)
FROM dividends
GROUP BY Ticker
ORDER BY Ticker;
```

```
SELECT Ticker, DivAmt, DivDate, SUM(DivAmt) OVER (PARTITION BY Ticker) total_div
FROM dividends;
```

```
SELECT Ticker, DivAmt, DivDate, SUM(DivAmt) OVER (PARTITION BY Ticker) total_div,
row_number() OVER (PARTITION BY Ticker ORDER BY DivDate) as `row`
FROM dividends;
```



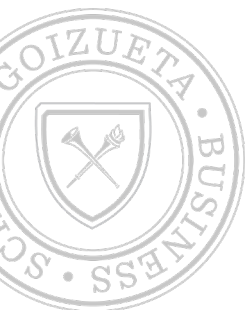
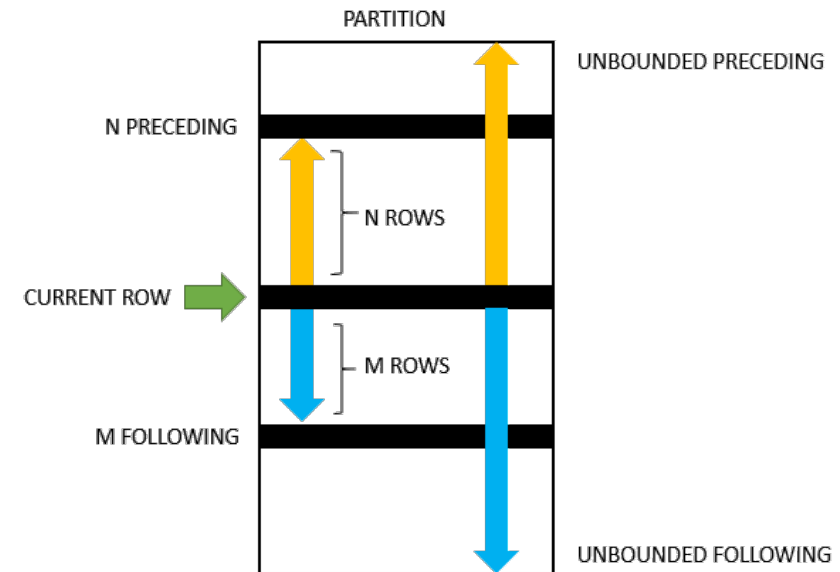
Source: <https://sqlite.org/windowfunctions.html>

SQL – Windows (BETWEEN)

- Define number of **following** rows to consider in moving window computation:

```
SELECT Ticker, DivAmt, DivDate, SUM(DivAmt) OVER  
(PARTITION BY Ticker ROWS BETWEEN CURRENT ROW AND  
UNBOUNDED FOLLOWING) total_div, row_number() OVER  
(PARTITION BY Ticker ORDER BY DivDate) as `row`  
FROM dividends;
```

```
SELECT Ticker, DivAmt, DivDate, SUM(DivAmt) OVER  
(PARTITION BY Ticker ROWS BETWEEN CURRENT ROW AND 2  
FOLLOWING) total_div, row_number() OVER (PARTITION BY  
Ticker ORDER BY DivDate) as `row`  
FROM dividends;
```

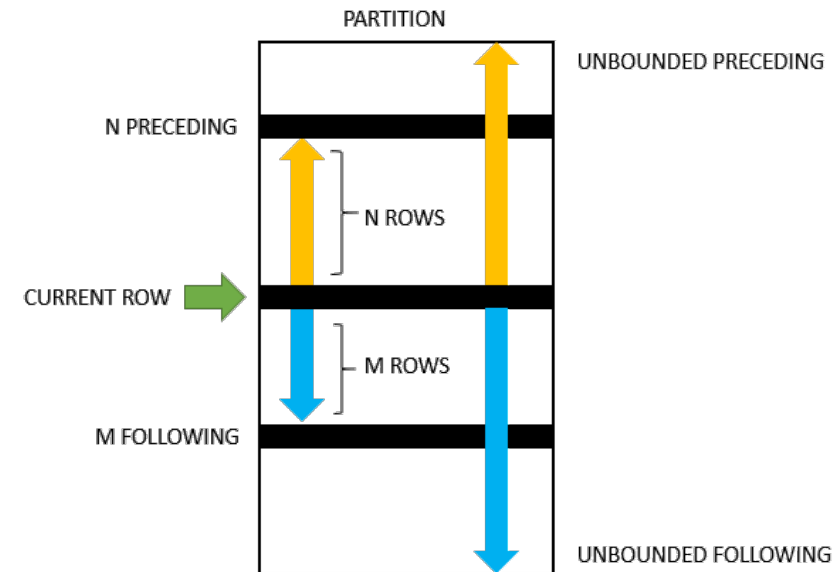


SQL – Windows (BETWEEN)

- Define number of **preceding** rows to consider in moving window computation:

```
SELECT Ticker, DivAmt, DivDate, SUM(DivAmt) OVER  
(PARTITION BY Ticker ROWS BETWEEN UNBOUNDED PRECEDING AND  
CURRENT ROW) total_div, row_number() OVER (PARTITION BY  
Ticker ORDER BY DivDate) as `row`  
FROM dividends;
```

```
SELECT Ticker, DivAmt, DivDate, SUM(DivAmt) OVER  
(PARTITION BY Ticker ROWS BETWEEN 2 PRECEDING AND CURRENT  
ROW) total_div, row_number() OVER (PARTITION BY Ticker  
ORDER BY DivDate) as `row`  
FROM dividends;
```



SQL – Windows Comparison (BETWEEN)

- Comparing results from unbounded following and unbounded preceding:

```
SELECT Ticker, DivAmt, DivDate,  
SUM(DivAmt) OVER (PARTITION BY Ticker ROWS BETWEEN  
CURRENT ROW AND UNBOUNDED FOLLOWING)  
total_div_fol,  
SUM(DivAmt) OVER (PARTITION BY Ticker ROWS BETWEEN  
UNBOUNDED PRECEDING AND CURRENT ROW)  
total_div_pre,  
row_number() OVER (PARTITION BY Ticker ORDER BY  
DivDate) as `row`  
FROM dividends;
```

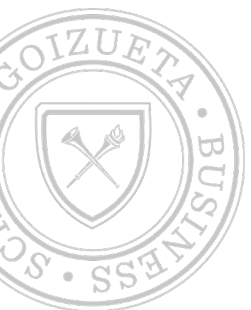
Ticker	DivAmt	DivDate	DivFol	DivPre	r
AACC	2.45	2007-07-17	2.45	2.45	1
AAON	0.13333	2006-06-08	1.10666	0.26666	1
AAON	0.13333	2006-12-11	1.23999	0.13333	2
AAON	0.13333	2007-06-07	0.97333	0.39999	3
AAON	0.16	2007-12-11	0.83999	0.55999	4
AAON	0.16	2008-06-10	0.67999	0.71999	5
AAON	0.16	2008-12-10	0.52	0.87999	6
AAON	0.18	2009-06-09	0.36	1.05999	7
AAON	0.18	2009-12-10	0.18	1.23999	8



SQL – Windows (RANGE)

- ROWS BETWEEN provides window function for rows in data. What if data is missing dates?
 - How do we use windows over dates and not rows?

```
SELECT Ticker, DivAmt, DivDate, SUM(DivAmt) OVER (PARTITION BY Ticker ORDER BY  
DATE(DivDate) RANGE BETWEEN INTERVAL 1 DAY PRECEDING AND INTERVAL 1 DAY FOLLOWING)  
date_sum  
FROM dividends;
```



SQL – LAG/LEAD

- What if you want to compare the values that are a lagging or leading?

```
SELECT Ticker, DivAmt, DivDate,  
lag(DivAmt, 1) OVER (PARTITION BY Ticker) lag_div,  
lead(DivAmt, 1) OVER (PARTITION BY Ticker) lead_div  
FROM dividends;
```

Ticker	DivAmt	DivDate	lag_div	lead_div
AACC	2.45	2007-07-17	NULL	NULL
AAON	0.18	2009-12-10	NULL	0.18
AAON	0.18	2009-06-09	0.18	0.16
AAON	0.16	2008-12-10	0.18	0.16
AAON	0.16	2008-06-10	0.16	0.16
AAON	0.16	2007-12-11	0.16	0.13333
AAON	0.13333	2007-06-07	0.16	0.13333
AAON	0.13333	2006-12-11	0.13333	0.13333
AAON	0.13333	2006-06-08	0.13333	NULL
AAPL	0.03	1995-11-21	NULL	0.03
AAPL	0.03	1995-08-16	0.03	0.03

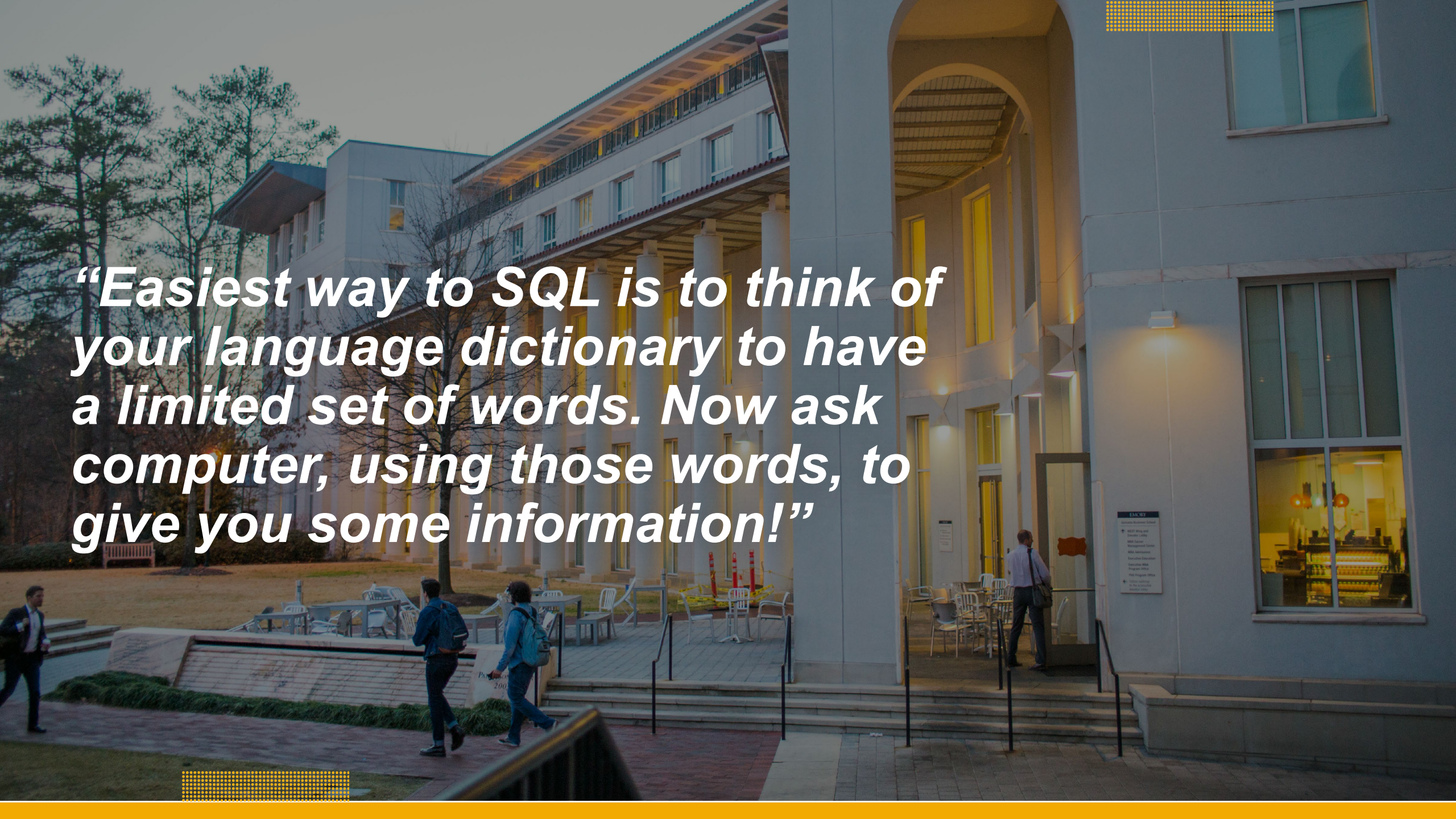


Reflect

Close your eyes, relax,
and reflect on ONE take
away from today's class.

5:00





“Easiest way to SQL is to think of your language dictionary to have a limited set of words. Now ask computer, using those words, to give you some information!”